1. Bubble Sort

Pseudocode

BubbleSort(A)
for i = 0 to n-1
 for j = 0 to n-i-2
  if A[j] > A[j+1]
   swap A[j] dengan A[j+1]

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

data = [5, 3, 8, 4, 2]
print("Bubble Sort:", bubble_sort(data))
```

```
Bubble Sort: [2, 3, 4, 5, 8]
```

Analisis Kompleksitas

Loop luar berjalan sebanyak n kali
Loop dalam berjalan sebanyak (n-1), (n-2), ..., 1

Total operasi:
(n-1) + (n-2) + ... + 1

= n(n-1)/2

Kompleksitas:
Best Case : O(n)
Worst Case : $O(n^2)$
Average : $O(n^2)$

2. Selection Sort

Pseudocode

SelectionSort(A)
for i = 0 to n-1
 min = i
 for j = i+1 to n
  if A[j] < A[min]
   min = j
 swap A[i] dengan A[min]

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

data = [5, 3, 8, 4, 2]
print("Selection Sort:", selection_sort(data))
```

```
Selection Sort: [2, 3, 4, 5, 8]
```

Analisis Kompleksitas

Loop luar berjalan n kali
Loop dalam membandingkan sisa elemen

Total operasi:
(n-1) + (n-2) + ... + 1

= n(n-1)/2

Kompleksitas:
Best Case : $O(n^2)$
Worst Case : $O(n^2)$
Average : $O(n^2)$

3. Insertion Sort

Pseudocode

InsertionSort(A)
for i = 1 to n-1
 key = A[i]
 j = i-1
 while j >= 0 dan A[j] > key
  A[j+1] = A[j]
  j = j - 1
 A[j+1] = key

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr

data = [5, 3, 8, 4, 2]
print("Insertion Sort:", insertion_sort(data))
```

```
Insertion Sort: [2, 3, 4, 5, 8]
```

Analisis Kompleksitas

Best Case (data sudah urut):
→ hanya 1 perbandingan tiap iterasi
→ O(n)

Worst Case (data terbalik):
1 + 2 + 3 + ... + (n-1)

= n(n-1)/2

Kompleksitas:
Best Case : O(n)
Worst Case : O(n^2)
Average : O(n^2)

4. Shell Sort

Pseudocode

```
ShellSort(A)
gap = n/2
while gap > 0
 for i = gap to n-1
  temp = A[i]
  j = i
  while j >= gap dan A[j-gap] > temp
   A[j] = A[j-gap]
   j = j - gap
  A[j] = temp
 gap = gap / 2
```

```
def shell_sort(arr):
    n = len(arr)
    gap = n // 2

    while gap > 0:
        for i in range(gap, n):
            temp = arr[i]
            j = i
            while j >= gap and arr[j-gap] > temp:
                arr[j] = arr[j-gap]
                j -= gap
            arr[j] = temp
        gap //= 2
    return arr

data = [5, 3, 8, 4, 2]
print("Shell Sort:", shell_sort(data))
```

```
Shell Sort: [2, 3, 4, 5, 8]
```

Analisis Kompleksitas

Shell Sort menggunakan gap (jarak)

Karena gap mengecil: jumlah perbandingan lebih sedikit

Kompleksitas:
Best Case : O(n log n)
Worst Case : O(n^2)
Average : sekitar O(n^1.3)